

Министерство науки и высшего образования
Пензенский государственный университет Кафедра
Вычислительная техника

ОТЧЕТ

по лабораторной работе N10

по курсу “Логика и основы алгоритмизации в инженерных задачах”

на тему “ Поиск расстояний во взвешенном графе”

Выполнили: студенты группы 24ВВВ2:

Воеводин И.А.

Гуляевский Д.С.

Каташов К.А.

Приняли:

Деев М. В.

Юрова О. В.

Пенза 2025

Задания:

Задание 1

1 Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G. Выведите матрицу на экран.

2 Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.

3.*

Сгенерируйте (используя генератор случайных чисел) матрицу смежности для ориентированного взвешенного графа G. Выведите матрицу на экран и осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием.

Задание 2

1 Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.

2 Определите подмножества периферийных и центральных вершин.

Задание 3*

1 Модернизируйте программу так, чтобы получить возможность запуска программы с параметрами командной строки (см. описание ниже). В качестве параметра должны указываться тип графа (взвешенный или нет) и наличие ориентации его ребер (есть ориентация или нет).

Примечание: задания, помеченные символом * выполняются по указанию преподавателя.

Описание метод решения

Программа решает задачу анализа взвешенного неориентированного графа, включая построение матрицы смежности, вычисление кратчайших расстояний между всеми вершинами, определение эксцентриситетов вершин, а также нахождение радиуса и диаметра графа. В основе метода лежит модифицированный алгоритм поиска в ширину (BFS), адаптированный для работы с взвешенными рёбрами. Сначала пользователь задаёт количество вершин графа, после чего генерируется случайная симметричная матрица смежности, где ненулевые значения соответствуют весам рёбер от 1 до 10, а нули — отсутствию рёбер. Главная диагональ заполняется нулями, что означает отсутствие петель. Для вычисления кратчайших путей из каждой вершины во все остальные используется функция BFSD, которая работает по принципу обхода графа в ширину, но с учётом весов рёбер: если обнаруживается путь к соседней вершине короче ранее найденного, расстояние обновляется, и эта вершина помещается в очередь для дальнейшего распространения волны. Этот подход является упрощённой версией алгоритма Дейкстры, где вместо приоритетной очереди используется обычная очередь, что может привести к лишним вычислениям, но сохраняет корректность результата для неотрицательных весов. После построения матрицы расстояний вычисляются эксцентриситеты вершин — максимальные расстояния от каждой вершины до всех остальных. На основе эксцентриситетов определяются радиус графа как минимальный эксцентриситет и диаметр как максимальный эксцентриситет. Если граф оказывается несвязным, то есть существуют недостижимые вершины, программа выводит соответствующее сообщение. Для выполнения задания с ориентированным графом необходимо изменить генерацию матрицы смежности, сделав её несимметричной, чтобы отражать направленность рёбер, после чего применить тот же алгоритм поиска расстояний, который уже корректно работает с ориентированными связями. Таким образом, метод решения охватывает генерацию графа, поиск кратчайших путей, анализ метрических характеристик и подготовку данных

для определения центральных и периферийных вершин, которые могут быть выделены путём сравнения эксцентриситетов вершин с радиусом и диаметром графа.

Листинг

```
#include <iostream>

#include <locale.h>

#include <time.h>

#include <cstdio>

#include <queue>

#include <climits>

#include <cstdlib>

using namespace std;

void BFSD(int** G, int* dist, int numG, int** GD) {

    for (int s = 0; s < numG; s++) {

        for (int i = 0; i < numG; i++) {

            GD[s][i] = INT_MAX;

        }

        GD[s][s] = 0;

        queue<int> q;

        q.push(s);

        while (!q.empty()) {

            int v = q.front();

            q.pop();

            for (int i = 0; i < numG; i++) {

                if (G[v][i] > 0 && GD[s][i] > GD[s][v] + G[v][i]) {

                    GD[s][i] = GD[s][v] + G[v][i];

                    q.push(i);

                }

            }

        }

    }

}
```

```
    }  
}  
}
```

```
int main() {  
    srand(time(NULL));  
    setlocale(LC_ALL, "rus");  
    int r = INT_MAX;  
    int d = -1;  
    int** G;  
    int** GD;  
    int* Ecc;  
    int numG;  
  
    std::cout << "Введите кол-во элементов: " << std::endl;  
    std::cin >> numG;  
  
    G = (int**)malloc(numG * sizeof(int*));  
    for (int i = 0; i < numG; i++) {  
        G[i] = (int*)malloc(numG * sizeof(int));  
    }  
  
    GD = (int**)malloc(numG * sizeof(int*));  
    for (int i = 0; i < numG; i++) {  
        GD[i] = (int*)malloc(numG * sizeof(int));  
        for (int j = 0; j < numG; j++) {  
            GD[i][j] = 0;  
        }  
    }  
  
    Ecc = (int*)malloc(numG * sizeof(int));  
  
    for (int i = 0; i < numG; i++) {  
        Ecc[i] = 0;  
    }  
}
```

```

for (int j = 0; j < numG; j++) {
    if (i == j) {
        G[i][j] = 0;
    }
    else {
        G[i][j] = (rand() % 2 == 0) ? 0 : (rand() % 10 + 1);
        G[j][i] = G[i][j];
    }
}
}

```

```

std::cout << "Матрица смежности графа:" << std::endl;

```

```

for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        printf("%3d ", G[i][j]);
    }
    printf("\n");
}

```

```

BFSD(G, Ecc, numG, GD);

```

```

std::cout << "\nМатрица расстояний:" << std::endl;

```

```

for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        printf("%3d ", GD[i][j] == INT_MAX ? -1 : GD[i][j]);
        if (GD[i][j] != INT_MAX && Ecc[i] < GD[i][j]) {
            Ecc[i] = GD[i][j];
        }
    }
    printf("\n");
}

```

```

std::cout << "\nЭксцентриситеты вершин:" << std::endl;

```

```

for (int i = 0; i < numG; i++) {

```

```

printf("Вершина %d: %d\n", i, Ecc[i]);

if (Ecc[i] > 0) {
    r = (r > Ecc[i]) ? Ecc[i] : r;
    d = (d < Ecc[i]) ? Ecc[i] : d;
}
}

if (r == INT_MAX || d == -1) {
    std::cout << "\nГраф несвязный!" << std::endl;
}
else {
    std::cout << "\nРадиус графа: " << r << std::endl;
    std::cout << "Диаметр графа: " << d << std::endl;
}

for (int i = 0; i < numG; i++) {
    free(G[i]);
    free(GD[i]);
}
free(G);
free(GD);
free(Ecc);

return 0;
}

```

Результат работы программы

Консоль отладки Microsoft Visual Studio

Введите кол-во элементов:

5

Матрица смежности графа:

0	0	6	3	0
0	0	10	8	4
6	10	0	0	0
3	8	0	0	0
0	4	0	0	0

Матрица расстояний:

0	11	6	3	15
11	0	10	8	4
6	10	0	9	14
3	8	9	0	12
15	4	14	12	0

Эксцентриситеты вершин:

Вершина 0: 15

Вершина 1: 11

Вершина 2: 14

Вершина 3: 12

Вершина 4: 15

Радиус графа: 11

Диаметр графа: 15

Вывод

В результате выполнения лабораторной работы были получены навыки работы с взвешенными и не взвешенными, а также ориентированными и не ориентированными графами. В ходе работы был сгенерирован граф заданного размера с случайными весами рёбер, построена матрица смежности и вычислены кратчайшие расстояния между всеми парами вершин. На основе полученной матрицы расстояний определены эксцентриситеты вершин, что позволило найти радиус и диаметр графа. Алгоритм корректно обрабатывает взвешенные рёбра и выявляет несвязность графа, предупреждая о ней в случае недостижимых вершин. Предложенный подход может быть легко адаптирован для ориентированного графа путём изменения генерации матрицы смежности на несимметричную, поскольку алгоритм поиска расстояний уже поддерживает работу с направленными связями. Таким образом, программа решает поставленные задачи, предоставляя инструмент для анализа основных метрических характеристик графов, что является важным этапом в изучении их структуры и свойств.